

136. La musique d'un jeu

Tout ensemble : un lutin qu'on déplace, une pièce à attraper, et un air qui ne s'arrête jamais.

1. Ce qu'il faut comprendre

Voici les dix niveaux réunis dans un seul programme : des tuiles dessinées, des lutins au pixel près, des boutons lus à chaque image, un score — et **deux airs**.

`jouer(1, MUSIQUE, 8, 1)` lance la musique de fond ; le quatrième argument, `1`, la fait **recommencer sans fin**. Ensuite le programme n'a plus rien à tenir : l'air avance tout seul dans l'interruption de la console, soixante fois par seconde, **même quand le jeu est en retard**.

Attraper la pièce lance `jouer(1, VICTOIRE, 5)` : la fanfare **prend la voix 1**, par-dessus la musique, exactement comme dans un vrai jeu. Elle, elle ne boucle pas.

`airFinis(1)` dit quand la fanfare est arrivée au bout. C'est là qu'on relance la musique de fond — sans cela, la voix resterait muette après la victoire, et l'on chercherait longtemps pourquoi.

La **partition ci-dessous** écrit les deux airs à la souris, et « Écouter » les fait entendre sans compiler. Change une note : le programme se réécrit, la cartouche se refait, et la console joue ta version dans la seconde.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Air>` → [53.1. Air — écrire une musique](#)
- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <jouer>` → [53.2. La fonction jouer\(\) — jouer un air tout seul](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <airFinis>` → [58.1. La fonction airFinis\(\) — l'air est-il fini ?](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1821 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      90 o ← la console, à l'allumage — la mise en route de la console
main()         338 o ← démarrage — ta fonction
VBlank         11 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AvancerAirs    308 o ← VBlank — fait avancer la musique à chaque image
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins   11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage — installe la copie des lutins en mémoire rapide
```

```

CopierTuiles      18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            736 o ← CopierTuiles – la police entière, et 2 tuiles dessinées
DonneesTransfert  10 o ← InstallerTransfert – la petite routine qui copie les lutins
Notes             120 o ← AvancerAirs – la table des notes de musique
"SCORE"           5 o ← texte() – un texte gravé
MUSIQUE_0         32 o ← jouer() – une table gravée
VICTOIRE_1        10 o ← jouer() – une table gravée

```

Lettres gravées : toutes (poser()) reçoit un numéro calculé)

Pas gravé (jamais appelé) : AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau

Fonctions du langage appelées :

```

jouer()   × 3   158 o sur place, dans main() + MUSIQUE_0 (32 o), VICTOIRE_1 (10 o)
sprite() × 2    46 o sur place, dans main()
poser()  × 1    12 o sur place, dans main()
texte()  × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 2    10 o sur place, dans main() + LireManette (37 o)
image()  × 1     6 o sur place, dans main() + AttendreImage (28 o)
airFin() × 1     5 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

/* La musique tourne pendant que le jeu tourne. */

#include <Air>      // un air de musique, note par note
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <jouer>    // joue un air tout seul
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près
#include <poser>    // pose une tuile sur une case du fond
#include <airFin>   // dit si un air est fini

```

2. UNE MÉLODIE — MUSIQUE

```

Air MUSIQUE = {
    "D04 10", "==", "SOL4 10", "==", "MI4 10", "==", "SOL4 10", "==",
    "FA4 10", "==", "LA4 10", "==", "SOL4 10", "==", "--", "--",
};

```

3. UNE MÉLODIE — VICTOIRE

```

Air VICTOIRE = {
    "D05 14", "MI5 14", "SOL5 15", "==", "--",
};

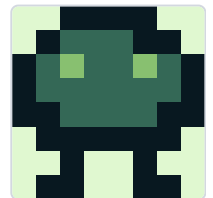
```

4. UN DESSIN — HEROS

```

Tuile HEROS = {
    "00333300",
    "03222330",
    "32122123",
    "32222223",
    "33222233",
    "03333330",
    "00300300",
    "03300330",
};

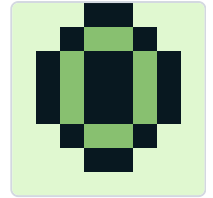
```



à l'écran

5. UN DESSIN — PIECE

```
Tuile PIECE = {  
    "00033000",  
    "00311300",  
    "03133130",  
    "03133130",  
    "03133130",  
    "00311300",  
    "00033000",  
    "00000000",  
};
```



à l'écran

6. UNE VARIABLE — X

```
uint8_t x = 40;
```

7. UNE VARIABLE — PIECE

```
uint8_t piece = 60;
```

8. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

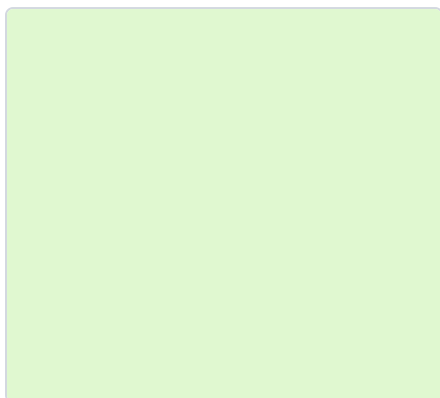
9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(1, 1, "SCORE");  
    jouer(1, MUSIQUE, 8, 1);  
  
    while (true) {  
        image();  
  
        if (bouton(DROITE) && x < 152) x++;  
        if (bouton(GAUCHE) && x > 8) x--;  
  
        sprite(0, x, 80, HEROS);  
        sprite(1, piece, 80, PIECE);  
  
        /* Attrapée ? On avance le score, on joue la fanfare par-dessus la  
        musique, et la pièce repart plus loin. */  
        if (x + 6 > piece && piece + 6 > x) {  
            if (score < 9) score++;  
            poser(7, 1, 27 + score);  
            jouer(1, VICTOIRE, 5);  
            piece = piece + 40;  
            if (piece > 150) piece = 20;  
        }  
  
        /* La fanfare finie, la musique reprend là où elle en était : au début. */  
        if (airFini(1)) jouer(1, MUSIQUE, 8, 1);  
    }  
  
    return 0;  
}
```

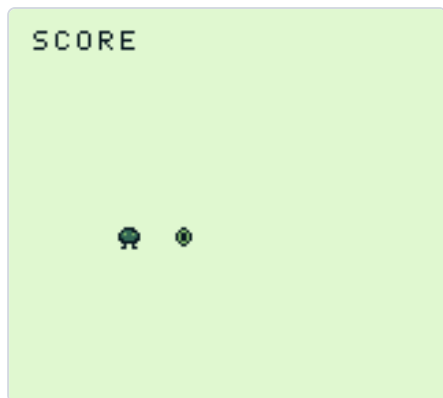
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est

la vérité sur ce programme.



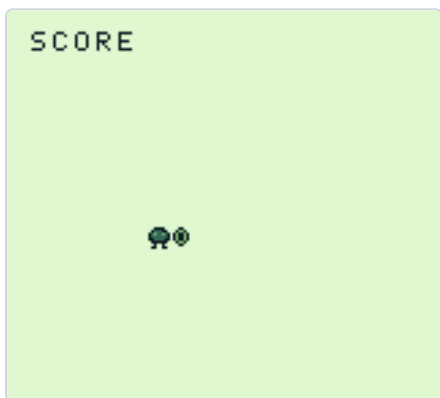
la toute première image — la console vient de démarrer — image 1



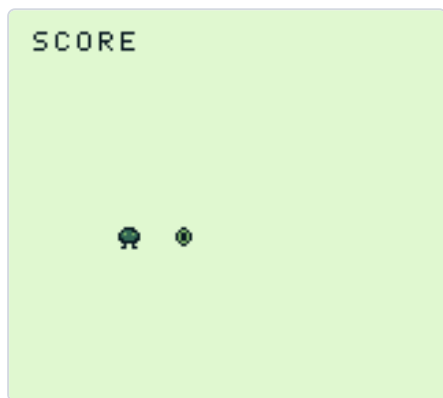
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

DROITE et GAUCHE déplacent le héros. La pièce attrapée fait monter le score et sonner une fanfare.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le héros est un lutin, hors de la grille
- ✓ la musique tourne dès le départ (329, 392 Hz)
- ✓ DROITE le déplace au pixel près (de 40 à 80)
- ✓ la pièce attrapée fait monter le score (score 1)
- ✓ et la fanfare sonne par-dessus, plus aigu (524, 659, 785 Hz)

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

MUSIQUE

VICTOIRE

+ air

MIDI

WAV

MUSIQUE

Écouter

note

- silence

= tenir

▼

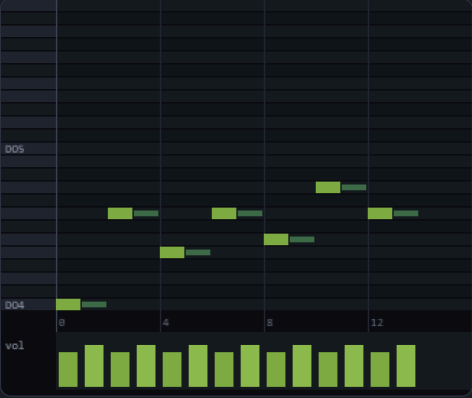
▲

- pas

+ pas

8

images par pas



The interface displays a piano roll with a grid of 12 horizontal lanes. The bottom lane is labeled 'vol' (volume). A series of green rectangular notes are placed on the grid, with their horizontal position corresponding to the volume bar below. The volume bar consists of 12 vertical bars of varying heights. The notes are placed on the 4th, 5th, 6th, 7th, 8th, 9th, 10th, 11th, and 12th lanes. The volume bar has 12 bars, with the first bar being the tallest and the last bar being the shortest.

`jouer(1, MUSIQUE, 8);`

— le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

Ensuite : **137. Le monde glisse, le héros ne bouge pas** — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche Game Boy en C++